10.9 Disjoint Set Union (Union-Find)

Disjoint Set Union, commonly known as Union-Find, is a data structure that manages a partition of a set into disjoint subsets. This structure supports two primary operations: finding the representative (or leader) of the subset containing a particular element, and uniting two subsets into a single subset. These operations are essential in many applications such as network connectivity, image processing, and is an integral part of algorithms like Kruskal's minimum spanning tree algorithm.

The primary goal of the Union-Find data structure is to perform these operations efficiently. To achieve this, Union-Find uses two main techniques: Union by Rank and Path Compression.

Union by Rank: This technique ensures that the tree representing each subset remains shallow. Each node is assigned a rank, which approximates the height of the tree rooted at that node. When performing the union operation, the root of the tree with a lower rank is made a child of the root with a higher rank. This keeps the resulting tree balanced, preserving efficiency.

Path Compression: This technique flattens the structure of the tree whenever the find operation is used. After finding the root of the tree, the nodes visited during the find operation are directly connected to the root. This significantly improves the efficiency of subsequent operations by reducing the height of the trees.

Implementation

Below is the implementation of the Union-Find data structure with Union by Rank and Path Compression in Python.

```
class UnionFind:
   def __init__(self, size):
      self.parent = list(range(size))
      self.rank = [0] * size

   def find(self, node):
      if self.parent[node] != node:
         self.parent[node] = self.find(self.parent[node]) # Path compression
      return self.parent[node]

   def union(self, node1, node2):
      root1 = self.find(node1)
      root2 = self.find(node2)

      if root1 != root2:
         if self.rank[root1] > self.rank[root2]:
            self.parent[root2] = root1
         elif self.rank[root1] < self.rank[root2]:
            self.parent[root1] = root2
         else:
            self.parent[root2] = root1
            self.rank[root1] += 1
```

In this implementation:

- The `__init__` function initializes two lists: `parent`, where each node is initially its own parent, and `rank`, which starts with all zeros.
- The `find` function uses path compression by recursively setting the parent of each node to the root of the tree.

- The `union` function connects two trees by comparing their ranks and attaching the shorter tree to the root of the taller tree. If both trees are of the same rank, one becomes the root and its rank is incremented.

Complexity Analysis

The time complexity of both the `find` and `union` operations can be analyzed using the inverse Ackermann function, denoted as $\alpha(n)$. This function grows extremely slowly, so for all practical purposes, it can be considered nearly constant. Therefore, both operations effectively run in amortized constant time, $\mathcal{O}(\alpha(n))$.

Applications

The Union-Find data structure is used in various applications, including:

- **Kruskal's Algorithm:** This algorithm for finding the minimum spanning tree (MST) of a graph heavily relies on Union-Find to detect cycles efficiently.
- **Connected Components:** In graph theory, Union-Find can determine if two vertices are in the same connected component.
- **Dynamic Connectivity:** This problem involves answering connectivity queries on a graph that changes over time, and Union-Find provides an efficient way to handle these updates.

Path Compression and Union by Rank Example

Consider a set with 7 elements: {0,1,2,3,4,5,6}. We perform the following operations:

1. Union(0, 1)
2. Union(1, 2)
3. Union(3, 4)
4. Union(4, 5)
5. Union(2, 5)
6. Find(5)
7. Find(6)

The sequence of operations and the effect of path compression can be visualized as:

```
uf = UnionFind(7)
uf.union(0, 1) # Union sets containing 0 and 1
uf.union(1, 2) # Union sets containing 1 and 2
uf.union(3, 4) # Union sets containing 3 and 4
uf.union(4, 5) # Union sets containing 4 and 5
uf.union(2, 5) # Union sets containing 2 and 5
print(uf.find(5)) # Finds the representative of the set containing 5
print(uf.find(6)) # Finds the representative of the set containing 6
```

This will result in:

```
0
6
```

Initially, each element is its own parent. After the unions, the internal structure with union by rank and path compression would be optimized, ensuring that subsequent operations are efficient. The element 5 is found under the representative element 0 after the `find(5)` operation.

By ensuring an efficient, practical implementation and understanding of the Union-Find data structure, it is possible to apply it to numerous problems requiring disjoint set management, optimizing both performance and readability.